QUANTUM 输入字典规范与实现方案

geometry.dict / mesh.dict / pic.dict / mcc.dict（ExplicitPIC 模块）

面向实现的设计报告（方法 + 代码落地）

2026-01-29

目录

1 背景与目标 **1**

2 总体设计原则（必须固定的规则） **2**

2.1 编号与引用规则 . . . 2

2.2 量纲与单位 . . . 2

2.3 domain（计算域方框）的唯一来源 . . . 2

3 geometry.dict 规范（固定格式） **2**

3.1 核心理念与数据结构 . . . 2

3.2 推荐的文件骨架（带说明的模板） . . . 3

3.3 关键语义说明 . . . 7

3.4 Phase 并集扩展（可选） . . . 8

4 mesh.dict 规范（BackgroundMesh + QuadtreeMesh + 自适应驱动声明） **8**

4.1 目标 . . . 8

4.2 推荐模板 . . . 8

4.3 几何量驱动的实现方案（边界-相解耦） . . . 10

4.4 物理量驱动的通用对接方案 . . . 10

5 pic.dict 规范（物理参数 + BC + 物理量驱动参数） **11**

5.1 边界条件：从“侧面 key”升级为“引用 Boundary ID”】【必须】 . . . 11

5.2 推荐模板（带单位） . . . 12

5.3 与现有代码能力的对应关系 . . . 14

5.4 碰撞/反应输入迁移原则（PIC vs MCC） . . . 14

6 mcc.dict 规范（碰撞/反应：MCC 子模块） **14**

6.1 职责与边界 . . . 14

6.2 推荐模板（带单位） . . . 15

6.3 关键语义（建议固定） . . . 16

7 代码实现方案（按模块分解） 17
7.1 geometry.dict 解析与数据模型 17
7.2 LevelSet 生成（从 Line 集合到 signed distance） 18
7.3 Phase 场的构造与检查 18
7.4 边界到 BC 的映射与施加 19
7.5 mesh 自适应：几何量 + 物理量 19

8 任务分解（建议里程碑） 19

9 建议的 AI 对话驱动指令（逐步实现用） 20

1 背景与目标

当前 QUANTUM 的 ExplicitPIC 模块已具备 paperA 所需的主要算法链路(leaf-based quadtree、FV Poisson、EB cut-cell 以及 matched deposit/gather)。但在 **输入字典**层面，存在如下问题：

- **同一物理/几何概念在多个地方重复定义**（例如 domain 边界既可能隐含为四条边，也可能部分出现在 geometry.dict)。
- **边界(boundary)缺乏统一的"基础设施"定义**：当前 pic.dict 的 BC 仍以 left/right/top/bottom 这种隐含侧面为主，EB 也只有单一 Embedded BC，难以覆盖多导体/多介质/移动边界等一般问题。
- **几何、网格、物理计算定义未严格分离**：几何细分准则、物理量驱动自适应与 PIC 模块策略的对接不够清晰。
- **量纲输入不统一**：mesh.dict 已支持带单位坐标，但 geometry.dict 的点/线/innerPoint 等仍为无量纲形式。

本报告给出一套 **固定并可扩展**的输入字典规范，并给出对应的 **代码实现方案**。设计遵循你给出的三分法：

类别	字典文件	职责（单一来源）
几何定义	geometry.dict	点-线-水平集-相-边界（BC 施加的基础设施）
网格定义	mesh.dict	背景网格 + 四叉树 + 自适应驱动声明（不含物理参数）
物理计算（PIC）	pic.dict	物理参数 + BC（引用 geometry 的边界编号）+ 物理量驱动参数
碰撞/反应（MCC）	mcc.dict	碰撞截面/反应网络/数值策略（与 PIC 解耦，供 MCC 子模块使用）

run.dict 仅负责运行器选择与简单控制，本报告不讨论 run.dict。

2 总体设计原则（必须固定的规则）

2.1 编号与引用规则

- geometry.dict 中 **所有显式编号从 1 开始**（点、线、水平集、相、边界均如此）。代码读取后统一转换为 0-based 索引。
- pic.dict 与 mesh.dict 中引用 geometry 实体时，**只引用 geometry 的 "边界编号 / 相编号 / 水平集编号"**，不直接引用点编号。

2.2 量纲与单位

- 所有长度坐标允许使用 **(x unit, y unit)** 或 **(x, y) unit** 形式，例如 `(3 cm, 0 cm)` 或 `(3, 0) cm`。
- 单位换算由 pic.dict 的 `Units{...}` 决定（UnitsContext）。当 `mode=SI` 时，几何/网格输入的长度统一从 SI 转换到 code units。
- 若 `mode=CodeUnits`，则单位后缀可省略（默认已是 code units）。

2.3 domain（计算域方框）的唯一来源

- **domain 只在 geometry.dict 中定义**（`Domain{min/max}`）。
- mesh.dict 只给出背景网格的 `divide` 与四叉树控制参数；BackgroundMesh 的方框范围直接继承 geometry.dict 的 Domain。
- 代码启动时仅检查：geometry.dict 的 Domain 是否存在；若不存在则报错（ExplicitPIC 与 Poisson/FV 必需）。

3 geometry.dict 规范（固定格式）

3.1 核心理念与数据结构

geometry.dict 以 **点-线-水平集-相-边界**为主线：

1. **Point：**计算域四个角点 + 所有嵌入几何与 domain 的交点 + 线之间的交点均需显式列出并编号。
2. **Line：**由点编号定义的有向线段（straight/arc/spline）。Line 既是几何构造元，也是边界/水平集的 "拓扑骨架"。
3. **LevelSet：**由一组 Line（可带方向符号）组成，要求该线集合 **闭合**或 **连接 domain 边界从而分割 domain**，从而在整个 domain 上产生单值水平集函数 $\psi(x)$。
4. **Phase：**不直接等同于水平集。Phase 由一个或多个水平集经布尔运算（交/并/差/补）得到，并要求 **所有 phase 的集合填满 domain 且相互不交**。
5. **Boundary：**这是 BC 施加的唯一基础设施。边界分三类：domainBoundary / fixedBoundary / movingBoundary。

3.2 推荐的文件骨架（带说明的模板）

```
# geometry.dict (v3) - GEOMETRY + PHASE + BOUNDARY INFRASTRUCTURE
# All IDs start from 1.
# All coordinates may carry length units. If Units.mode=SI, they will be
# converted to code units using L0.

Domain
{
    # Rectangular computational domain.
    min (0 cm, 0 cm)
    max (3 cm, 6 cm)
}

Point
{
    # --- Domain corners (must be included) ---
    (0 cm, 0 cm) # id=1
    (3 cm, 0 cm) # id=2
    (3 cm, 6 cm) # id=3
    (0 cm, 6 cm) # id=4

    # --- Intersections on domain boundary (if any) ---
    # (x ?, y ?) # id=...

    # --- Embedded geometry key points ---
    # ...
}

Line
{
    # Each line is defined by two point IDs (start,end).
    # Optional: name for readability.

    (1, 2) # id=1 bottom edge of domain
    {
        name bottom
        type straight
    }

    (2, 3) # id=2 right edge
    {
        name right
        type straight
    }
```

```
    (3, 4) # id=3 top edge
    {
        name top
        type straight
    }

    (4, 1) # id=4 left edge
    {
        name left
        type straight
    }

    # --- Embedded boundaries (example) ---
    (5, 6) # id=5
    {
        name electrode_segment_1
        type straight
    }

    # arc:
    # (pA, pB)
    # {
    # type arc
    # center (xc cm, yc cm)
    # radius 1 mm
    # radian 0.5 # times pi
    # }

    # spline:
    # (pA, pB)
    # {
    # type spline
    # featurePoints ( ... ) # each point may carry units
    # }
}

LevelSet
{
    # A level set is built from a set of oriented lines.
    # A negative sign flips the line orientation (endpoints swapped).

    "LS_conductor"
    {
        lines ( +5, +6, -7, ... )
        innerPoint (x cm, y cm) # a point that must lie in the psi<0 side
        moving false
```

```
    }

    "LS_dielectric"
    {
        lines ( ... )
        innerPoint (x cm, y cm)
        moving false
    }
}

Phase
{
    # Phases are defined by boolean constraints on level sets.
    # A phase is an intersection of half-spaces by default.
    # Example: (LS_conductor < 0) AND (LS_dielectric > 0)

    "conductor" # id=1
    {
        type conductor
        constraints
        {
            ("LS_conductor" < 0)
        }
        boundaries ( 5, 6, 7 ) # OPTIONAL: boundary IDs that belong to this phase
    }

    "plasma" # id=2
    {
        type plasma
        constraints
        {
            ("LS_conductor" > 0)
            ("LS_dielectric" > 0)
        }
    }

    "dielectric" # id=3
    {
        type dielectric
        constraints
        {
            ("LS_dielectric" < 0)
            ("LS_conductor" > 0)
        }
        material
        {
```

```
            epsilon_r 4.0
        }
    }
}

Boundary
{
    # Boundary entities are the ONLY objects referenced by pic.dict for BC.
    # Each boundary has an ID and is defined by a set of Lines.
    # For fixed/moving boundaries, we must also specify which level set it
    # belongs to, and the phase on each side.

    domainBoundary
    {
        boundary 1
        {
            name left
            lines (4)
            adjacentPhase 2 # inside-domain phase ID
        }
        boundary 2
        {
            name right
            lines (2)
            adjacentPhase 2
        }
        boundary 3
        {
            name bottom
            lines (1)
            adjacentPhase 2
        }
        boundary 4
        {
            name top
            lines (3)
            adjacentPhase 2
        }
    }

    fixedBoundary
    {
        boundary 10
        {
            name cathode
            lines (5, 6, 7)
```

```
        # The geometric location is represented by the referenced level set.
        levelSetIndex 1 # refers to "LS_conductor" (1-based)

        # psi<0 side and psi>0 side phases
        phase 1
        opposedPhase 2

        immersed true
        ignored false
    }
}

movingBoundary
{
    boundary 20
    {
        name moving_meniscus
        lines (12, 13)
        levelSetIndex 2
        phase 2
        opposedPhase 3

        immersed true
        motion
        {
            model prescribed
            # example: translation in y
            velocity (0 mm/s, 1 mm/s)
        }
    }
}
}
```

3.3 关键语义说明

(1) LevelSet 的符号定义 对每个 LevelSet：

- `innerPoint` 必须处于 $\psi < 0$ 一侧。
- 若线集合闭合，$\psi < 0$ 侧可解释为 “内部”。若线集合连接 domain 边界并分割 domain，则 `innerPoint` 用于选定分割后的哪一侧为 $\psi < 0$。

(2) Phase 的定义与检查 Phase 使用 `constraints` 定义为多个半空间的交集：

$$\Omega_{\text{phase}} = \bigcap_i \{x : s_i\, \psi_{k_i}(x) < 0\}, \quad s_i \in \{+1, -1\}.$$

("LS" > 0) 等价于 $s = -1$。该形式覆盖常见的交/差构造（差：加一条 $\psi_B > 0$ 约束）。并集如确有需要，可通过 subregions 扩展（见 §3.4）。

启动时进行 **partition 检查**：在 BackgroundMesh 顶点采样（可选在 quadtree leaf 顶点采样），要求每个采样点 **恰好**属于一个 phase；否则报错并输出冲突位置。

(3) Boundary 的唯一性　pic.dict 只能引用 Boundary ID。Boundary 由 Lines 定义，但 BC 不再直接用 line index；这样可避免 line 与 BC 的耦合。

3.4 Phase 并集扩展（可选）

若需要并集（例如一个 phase 由多个不连通子域组成），在 Phase 中增加：

```
subregions
{
   region { constraints { ("LS1" < 0) ("LS2" > 0) } }
   region { constraints { ("LS3" < 0) } }
}
# overall region is UNION over all regions
```

实现上：每个 region 生成一个 signed indicator ϕ_r（交集用 max），总的并集用 $\phi = \min_r \phi_r$。

4 mesh.dict 规范（BackgroundMesh + QuadtreeMesh + 自适应驱动声明）

4.1 目标

mesh.dict 的职责是：

- 定义背景网格的分辨率（divide）与四叉树控制参数；**domain 来自 geometry.dict**。
- 定义 quadtree 的基本控制参数。
- 若 adaptiveMesh=true，则 **声明**需要哪些自适应驱动（几何/物理），但 **不放置**物理量的具体参数（参数在 pic.dict）。

4.2 推荐模板

```
# mesh.dict (v3)

BackgroundMesh
{
   divide (50, 100)
}

adaptiveMesh true

QuadtreeMesh
```

```
{
    maxCellNum 200000
    maxInitialIterations 3
    C_Lipschitz 2.6

    # ---------------------------
    # Geometry-driven refinement
    # ---------------------------
    GeometryRefinement
    {
        enabled true

        # Refine near specific boundaries (each may have its own maxLevel).
        # Use min_{v in V(K)} |psi_v| <= $C_\Gamma$ * h_K as trigger.
        # psi is the referenced level set (via boundary's levelSetIndex).
        refineBoundary
        {
            entry { boundaryId 10 $C_\Gamma$ 1.0 maxLevel 7 }
            entry { boundaryId 20 $C_\Gamma$ 1.0 maxLevel 8 }
        }

        # Phase interior level limits.
        refinePhase
        {
            phase 1 { minLevel 4 maxLevel 7 }
            phase 2 { minLevel 3 maxLevel 6 }
            phase 3 { minLevel 2 maxLevel 5 }
        }

        # NOTE: 本版本不单独暴露“按 cut-cell(0<alpha<1) 强制细化”的开关。
        # cut-cell 会随 boundary proximity refinement 自然出现并被细化;
        # 若未来确有需要, 可再加入一个全局 cap 作为兜底选项。
    }

    # ---------------------------
    # Physics-driven refinement
    # ---------------------------
    PhysicsRefinement
    {
        enabled true

        # Declare which indicators to query from physics modules.
        indicators ( DebyeLength ElectricField )

        # Generic policy knobs (module-independent)
        adaptEverySteps 20
```

```
        hysteresis 0.2
        persistenceSteps 3

        # Indicator-specific generic caps (optional)
        caps
        {
            DebyeLength { maxLevel 9 }
            ElectricField { maxLevel 8 }
        }
    }
}
```

4.3 几何量驱动的实现方案（边界-相解耦）

需求回顾 你希望把“每个水平集两侧 minLevel/maxLevel”的旧方式改为：

- **每条（或每组）嵌入边界可独立指定 max refinement level**。
- **每个相内部可独立指定 min/max level**。
- 触发准则以 $\min_{v\in\mathcal{V}(K)}|\psi_v| \le C_\Gamma h_K$ 为主（按边界邻近性细化）。

建议的落地方式

1. 由 geometry.dict 的 `Boundary.fixedBoundary / movingBoundary` 给出 boundaryId 与其 `levelSetIndex`。
2. mesh.dict 的 `GeometryRefinement.refineBoundary` 通过 boundaryId 选择要加密的边界对象，并给出该对象的 `maxLevel` 与 `C_Gamma`。
3. 在 quadtree 初始化与每次适配时，对每个 leaf cell K：
   - 计算该 cell 顶点上的 ψ（来自 ImmersedGeometry level set field）；
   - 若满足 $\min|\psi_v| \le C_\Gamma h_K$ 且 $\ell_K < \ell_{\max}(\text{boundary})$，则标记细分；
4. 相内部的 min/max level 作为 **硬约束**：
   - 若 cell K 完全属于相 p，且 $\ell_K < \ell_{\min}(p)$，则强制细分；
   - 若 $\ell_K > \ell_{\max}(p)$，则允许合并（与 Lipschitz/hysteresis 一起决定）。

4.4 物理量驱动的通用对接方案

目标 四叉树网格模块需要一个 **通用接口**对接不同物理模块的自适应量。mesh.dict 只声明“需要哪些量、由哪个模块提供”；每个物理模块在自己的 dict（对 ExplicitPIC 即 pic.dict）里提供该量的参数定义。

建议接口（C++） 在 Core/Mesh 或 Solvers/Mesh 下新增：

```
struct RefinementIndicatorField
{
    std::string name; // e.g. "DebyeLength"
    std::vector<double> cellValue; // value per leaf cell (code units)
};

class IMeshRefinementProvider
{
public:
    virtual ~IMeshRefinementProvider() = default;
    virtual std::string moduleName() const = 0; // "ExplicitPIC"
    virtual bool Provide(const std::string& indicatorName,
                         const QuadtreeMesh& mesh,
                         RefinementIndicatorField& out) = 0;
};
```

由各物理模块在初始化时注册 provider（例如通过全局 registry 或 App-level wiring）。

QuadtreeMeshRefiner 会向已注册的物理模块 provider 逐一查询所需 indicator（先到先得）。

指标到细化判据的统一 对于每个 indicator，网格侧仅做“通用”判断：

- **DebyeLength：** 目标是 $h_K \le C_D\,\lambda_D$。
- **ElectricField：** 目标是捕捉高梯度区域，可用 $|\nabla\phi|$ 或 $|E|$ 与阈值/分位数比较。

具体参数（例如 C_D、T_e、n_e floor、阈值定义）均在 pic.dict 中定义。

5 pic.dict 规范（物理参数 + BC + 物理量驱动参数）

5.1 边界条件：从“侧面 key”升级为“引用 Boundary ID”】【必须】

当前 pic.dict 的 `BC/Domain{left/right/...}` 方式默认 domain 边界为四条侧面，这与“几何显式定义边界”的目标冲突。新规范如下：

- pic.dict 的 BC 统一写在 `BC{...}` 下。
- 每条需要施加 BC 的边界通过 **boundaryId** 引用 geometry.dict 的 Boundary 实体。
- BC 类型限定为当前代码已支持的 Poisson BC：Dirichlet / Neumann / Robin（以及时间依赖 Dirichlet 作为扩展）。
- 允许同时定义 **Field BC** 与 **Particle BC**（例如吸收/反射/SEE），以支撑 PIC 需要。

5.2 推荐模板（带单位）

```
// pic.dict (v3)

PIC
{
   dt 5e-11 s
   steps 2000
   seed 12345
   omp 8
}

Units
{
   mode SI
   L0 3 cm
   T0 1 ns
   Phi0 800 V
   n0 1e16 1/m^3
   writeSIOutputs true
}

Solvers
{
   PoissonCellCentered { useSuperLU true useCG true useIC true maxIter 2000 relTol 1e
       ↪ -10 }
}

BC
{
   # ---- Poisson / electrostatic BCs ----
   Field
   {
      boundary 1 { type Dirichlet value 0 V } # left
      boundary 2 { type Dirichlet value 800 V } # right
      boundary 3 { type Neumann value 0 } # bottom
      boundary 4 { type Neumann value 0 } # top

      boundary 10 { type Dirichlet value 0 V } # cathode (embedded)
   }

   # ---- Particle-wall interaction ----
   Particle
   {
      boundary 10
      {
```

```
            model absorb
            # optional: SEE model by reusing existing SEEConfig keys
            SEE { enabled false }
        }
    }
}

Phases
{
    # Tell PIC which phase is plasma (where particles live)
    plasmaPhase 2

    # Solid phases: particles entering these phases are removed
    solidPhases (1, 3)
}

RefinementIndicators
{
    # Parameters used by mesh PhysicsRefinement.indicators

    DebyeLength
    {
        enabled true
        CD 0.5
        Te 2.0 eV
        neFloor 1e12 1/m^3
    }

    ElectricField
    {
        enabled false
        # example threshold policy
        percentile 95
        smoothWindowSteps 20
    }
}

Deposition
{
    useMatchedWeights true
    compositeBand true
    momentExactCut true
    cutRenorm true
}

Species
```

```
{
   "electron" { q -1 e m 9.1093837015e-31 kg Te 2.0 eV drift (0, 0) numParticles 20000
       ↪ weight 1e6 }
   "Ar_plus" { q 1 e m 39.948 amu Te 0.05 eV drift (0, 0) numParticles 20000 weight 1e6
       ↪  }
}

Output
{
   every 200
   fields ( phi rho E n )
   legacySeparate false
   console.level info
   console.every 50
}
```

5.3 与现有代码能力的对应关系

- **Field BC** 直接映射到 `Solvers/Poisson/PoissonBC` 目前支持的类型（Dirichlet/Neumann/Robin）。
- **Particle BC**：当前 ExplicitPIC 已支持 `absorbParticles`（对 embedded solid）。新规范把它提升为 “按 boundaryId 指定”。第一阶段实现可先支持 `absorb`，并复用现有 SEEConfig（仍可按 boundaryId 选择发射边界）。

5.4 碰撞/反应输入迁移原则（PIC vs MCC）

为避免 pic.dict 过于庞杂，并保持 “PIC 推进” 与 “碰撞/反应” 的解耦，本规范将所有与**碰撞**、**反应**、**截面**、**反应网络**、**数值碰撞算法**相关的输入**全部移出** pic.dict，单独形成 `mcc.dict`。pic.dict 仅保留：时间步/物种/沉积采样/Poisson 求解/边界条件/（可选）物理量驱动参数。

6 mcc.dict 规范（碰撞/反应：MCC 子模块）

6.1 职责与边界

`mcc.dict` 的职责是定义：

- 体碰撞模型（Monte Carlo Collision / Null-collision / DSMC-like 变体等）；
- 气体背景（组分、温度、压强或数密度）与中性粒子假设；
- 截面模型（解析式/表格数据）与反应通道（弹性/激发/电离/电荷交换/复合等）；
- 数值策略（子循环、最大碰撞频率估计、采样与统计输出）。

不在 mcc.dict 中定义边界条件（边界相互作用仍属于 pic.dict 的 Particle BC），也不重复定义几何/网格。

6.2 推荐模板（带单位）

```
# mcc.dict (v1) - Monte Carlo Collisions / Reactions

MCC
{
    enabled true

    # Algorithm options
    method NullCollision # NullCollision | DirectSampling
    subcycles 1 # collisions per PIC step (>=1)
    seed 2026

    # Safety / performance
    maxNu 5e9 1/s # optional cap for collision frequency
    rebuildNuEverySteps 50
}

BackgroundGas
{
    # One may specify pressure+temperature, or directly number density.
    model IdealGas
    Tgas 300 K
    p 50 Pa

    # Gas species composition (mole fractions)
    composition
    {
        Ar 1.0
        # N2 0.1
    }
}

# Map PIC species names (from pic.dict/Species) into collision participants.
Participants
{
    charged ( electron Ar_plus )
    neutral ( Ar )
}

CrossSections
{
    # Define cross sections by pair and process name.
    # Supported sources:
    # (1) analytic: hardSphere / powerLaw / userPolynomial
    # (2) tabulated: file with (E[eV], sigma[m^2])
```

```
    pair "electron"-"Ar"
    {
        elastic
        {
            source tabulated
            file "cross_sections/e_Ar_elastic.csv"
            energyUnit eV
            sigmaUnit m^2
        }

        ionization
        {
            source tabulated
            file "cross_sections/e_Ar_ionization.csv"
            threshold 15.76 eV
            products ( "electron" "electron" "Ar_plus" )
        }
    }
}

Reactions
{
    # Optional explicit reaction list if the code prefers a reaction graph.
    # Each reaction references a CrossSections entry.
    reaction 1
    {
        name e_Ar_ionization
        reactants ( "electron" "Ar" )
        products ( "electron" "electron" "Ar_plus" )
        process ionization
    }
}

Output
{
    every 200
    diagnostics ( nuMax nuMean reactionRates )
}
```

6.3 关键语义（建议固定）

- 截面自变量默认使用**相对动能**（例如电子-中性碰撞用电子在中性参考系下的能量，单位 eV）。
- 反应通道由 `pair+process` 唯一标识；`Reactions` 仅用于显式列举/重命名/输出统计。

- 若采用 NullCollision，需要维护每个 pair 的 $\nu_{\max}$；可由 maxNu 或重建策略控制。
- 所有文件路径均相对 case 目录；建议放入 case/cross_sections/ 子目录。

7 代码实现方案（按模块分解）

7.1 geometry.dict 解析与数据模型

现状与问题 现有 DictionaryGeometry 已能解析 Point/Line/LevelSet/Phase/Boundary，但存在：

- 点坐标与 innerPoint 不支持单位；featurePoints 也不支持单位。
- 边界仍是“按 line 标记”，缺少 boundaryId/group 的一等对象。
- Phase 的布尔运算语法不清晰，且输入中的 [newIndex] 并未被解析。

改造目标

1. **新增并固定 Domain 块**：domain 只在 geometry.dict 中定义（唯一来源）。
2. **统一 1-based 编号输入**。
3. **使用 UnitsContext 解析带单位坐标**：Point/innerPoint/arc center/spline featurePoints。
4. **引入 BoundaryEntity**：domain/fixed/moving 三类边界都以 boundaryId 为主键，并记录 lines、levelSetIndex、相邻 phase 信息。
5. **Phase 定义升级为 constraints/subregions**（替代旧的 levelSetIndexA/B + operation 链），实现更直观。

建议的数据结构（新增）

```
enum class BoundaryKind { Domain, Fixed, Moving };

struct BoundaryEntity
{
    int id = -1; // 0-based internal
    std::string name;
    BoundaryKind kind;
    std::vector<int> lineIds; // 0-based line indices

    // For Fixed/Moving
    int levelSetIndex = -1; // which LS defines this interface
    int phase = -1; // psi<0 side
    int opposedPhase = -1; // psi>0 side
    bool immersed = true;
    bool ignored = false;

    // For Domain
```

```
    int adjacentPhase = -1;
};

struct PhaseEntity
{
    int id = -1;
    std::string name;
    std::string type; // plasma|conductor|dielectric|...

    // Union of subregions; each subregion is an intersection of constraints.
    struct Constraint { int ls; int sign; }; // sign=+1 for (<0), sign=-1 for (>0)
    std::vector<std::vector<Constraint>> subregions;
};
```

7.2 LevelSet 生成（从 Line 集合到 signed distance）

目标 给定一组 line（straight/arc/spline），构造整域定义的 $\psi(x)$。要求：

- $\psi(x)$ 为到界面线集合的 signed distance（或其近似）。
- 符号由 `innerPoint` 选定：$\psi(\text{innerPoint}) < 0$。

实现建议（兼容现有逻辑） 第一阶段沿用当前 ImmersedGeometry 以网格顶点采样 ψ 的方式：

1. 对每个网格顶点 v，计算到所有 interface line segments 的最小距离 $d(v)$（arc/spline 可离散为足够密的 polyline）。
2. 判定符号：
   - 若线集合闭合：用 winding number 或 ray casting 判断点在内部/外部；
   - 若线集合分割 domain（与 domainBoundary 连接）：用 innerPoint 所在连通域作为 $\psi < 0$ 一侧（可通过在顶点图上 flood-fill 近似）。
3. 令 $\psi(v) = \pm d(v)$，并保留现有 reinitialize / updateLevelSet（若 moving=true）。

第二阶段可把 cut-cell metric (α, β, γ) 的几何计算从 “采样” 升级为 “基于 line 的解析切割”，但这不影响字典规范。

7.3 Phase 场的构造与检查

给定每个 level set 的 $\psi_k(v)$，对每个 phase p：

1. 对每个 subregion r（约束集合）构造
$$\phi_r(v) = \max_i \big(s_i\, \psi_{k_i}(v)\big).$$
2. phase 的 indicator 为并集：$\phi_p(v) = \min_r \phi_r(v)$。
3. 顶点归属：$v \in \text{phase } p \iff \phi_p(v) < 0$。

然后做 partition 检查（每个采样点恰属一个 phase）。

7.4 边界到 BC 的映射与施加

核心变化 从 “side key / 单一 embedded BC” 升级为 “boundaryId -> BC”。需要两条链：

1. **Domain boundary faces**：quadtree 的外边界 face 需要能查询其所属 boundaryId。
2. **Embedded boundary segments**：cut-cell 的 EB segment 需要能查询其所属 boundaryId（通常由其 levelSetIndex + interface line id 决定）。

第一阶段（最小改动，先跑通）

- domainBoundary：仍映射到 west/east/south/north 四侧，但由 geometry.dict 提供 boundaryId，对应 pic.dict 的 BC/Field/boundary `id`。
- fixedBoundary：先支持 “单一 embedded boundary”（即 pic.dict 指定一个 boundaryId 作为 EB），等价于现在的 EmbeddedGeometry.levelSetIndex + BC/Embedded。这样先完成 “BC 引用 boundaryId” 的闭环。

第二阶段（支持多固定边界、多 BC）

- 在 cut-cell metric/segment 构建时，记录每个 EB segment 属于哪个 levelSet，并进一步通过最近 line（或线段参数）映射到 boundaryId。
- PoissonFVSolverCellCentered 组装时，对每个 EB segment 调用 `bcMap.get(boundaryId)` 获取 Dirichlet/Neumann/Robin 参数。

7.5 mesh 自适应：几何量 + 物理量

几何量驱动 实现位置：`DictionaryMesh` 增加 `GeometryRefinement` 解析；`QuadtreeMesh` 的 refine 标记阶段增加 boundaryId/phase 逻辑。

物理量驱动 实现位置：新增 `IMeshRefinementProvider` 接口 + provider registry；ExplicitPIC 提供 DebyeLength 与 ElectricField 指标。

8 任务分解（建议里程碑）

阶段	交付物（可验收）	主要改动文件（建议）
M1	geometry.dict(v3) 与 mesh.dict(v3) 解析完成，支持单位与 1-based 编号；domain 只在 geometry.dict 定义；更新 electrospray 样例可读入	DictionaryGeometry.(h/cpp), Dictionary.(h/cpp), Line/Structures, ElectrosprayApp 启动链
M2	geometry.dict(v3) 的 BoundaryEntity 生效；pic.dict(v3) 的 Field BC 可按 boundaryId 施加到 domain 四侧（替代 left-/right）	DictionaryPIC, PicConfig(BC), PoissonFVSolverCellCentered, domain bc query

M3	mesh.dict(v3) 的 GeometryRefinement 生效（按 boundaryId maxLevel + phase min/max）；更新现有 ExplicitPIC cases 通过	DictionaryMesh, QuadtreeMesh refine logic, ImmersedGeometry phase labeling
M4	物理量驱动对接：mesh.dict 声明 DebyeLength；pic.dict 提供参数；ExplicitPIC provider 输出 indicator；quadtree 根据 h<=CD*lambdaD 细化	IMeshRefinementProvider, registry, ExplicitPIC provider, mesh refiner
M4b	引入 mcc.dict(v1) 解析与 MCC 子模块骨架：读取气体背景/截面/反应通道；与 ExplicitPIC 物种名对齐；先实现“空碰撞(no-op)”可开关运行	DictionaryMCC, MCCConfig, ExplicitPIC-MCC hook (stub)
M5	多固定边界 BC（多个 conductor/insulator）：cut segments 绑定 boundaryId；Poisson EB closure 按 boundaryId 取 BC	CutCellMetrics/segment builder, Poisson EB closure, geometry-boundary mapping

9 建议的 AI 对话驱动指令（逐步实现用）

下面给出可以直接复制给 AI 的“分步实现指令”，每一步都要求输出：修改点、补丁（diff）、以及能跑的最小验证 case。

1. **M1 指令：**

   请在 QUANTUM2 中实现 geometry.dict(v3) 的 Units 支持与 1-based 编号：(1) 给 DictionaryGeometry::readDict 增加 UnitsContext* 参数并用 parseCoordWithUnits/parseCoordsWithUnits 解析 Point/innerPoint/arc center/spline featurePoints；(2) 支持 Domainmin/max；(3) Point/Line/LevelSet/Boundary 的编号输入从 1 开始，内部转换为 0-based；(4) mesh.dict 的 BackgroundMesh 仅含 divide；BackgroundMesh 的 domain 直接继承 geometry.dict 的 Domain；若 Domain 缺失则报错。输出：代码 diff + 至少一个更新后的 case(electrospray 或 `explicitpic_two_plates_eb_post`)可正常读取并打印解析结果。

2. **M2 指令：**

   请把 ExplicitPIC 的 BC 从 left/right/top/bottom 迁移为引用 geometry Boundary ID：(1) 设计 PicConfig::FieldBCMap（boundaryId->PicBC）；(2) 更新 DictionaryPIC 解析 BC/Field/boundary <id> ...；(3) 在 PoissonFVSolverCellCentered 中，对 domain 外边界 face 通过 geometry 的 domainBoundary 映射到 boundaryId，再取对应 BC。保留旧语法兼容（若未给 boundaryId，则回退到旧 Domainleft...）。输出：代码 diff + `explicitpic_two_plates_eb_post` case 使用 boundaryId BC 运行通过。

3. **M3 指令：**

请实现 mesh.dict(v3) 的 GeometryRefinement：(1) DictionaryMesh 解析 GeometryRefinement/refineBoundary/refinePhase；(2) quadtree refine 阶段按 boundaryId 的 maxLevel + C_{Γ} 触发，以及 phase 的 min/max 约束。输出：代码 diff + 一份输出日志（每次 adapt 统计每个触发器贡献的细化数量）。

4. **M4 指令：**

请实现物理量驱动自适应的通用对接：新增 IMeshRefinementProvider 接口与 registry；ExplicitPIC 提供 DebyeLength 指标（用电子密度估计 ne，Te 可由 pic.dict 或 species 提供）；mesh 侧用 h<=CD*lambdaD 作为 refine 准则。输出：代码 diff + 一个对比实验（启用/禁用 DebyeLength 适配）展示网格等级变化。

5. **M5 指令：**

请扩展 EB 边界到多 boundaryId：cut-cell segment 构建时记录 boundaryId；Poisson EB closure 对每个 segment 查询对应 BC；pic.dict 支持为多个 boundaryId 设置不同 Dirichlet 值。输出：代码 diff + 一个包含两个导体的 EB case（两个导体不同电势）验证电势分布。

附注：与现有源码的对应入口

- geometry 解析：`src/Core/IO/DictionaryGeometry.cpp`
- mesh 解析：`src/Core/IO/DictionaryMesh.cpp`
- pic 解析：`src/Modules/Plasma/ExplicitPIC/DictionaryPIC.cpp`
- Poisson BC：`src/Solvers/Poisson/PoissonFVSolverCellCentered.cpp`
- EB 耦合与 cut metrics：`.../CutCellMetrics`，`FaceSegmentGraph`，`ElectricFieldEvaluator`